2D Embeddings for Length Generalization in Character-Based Transformers (CBT)

Neithan Vargas Vargas
neithanvarvar@gmail.com

Abstract

Small autoregressive Transformers trained from scratch on subword (BPE) tokens with one-dimensional (1D) positional embeddings fail on multi-digit addition beyond the digit lengths seen in training; in our experiments, nine 1D/NoPE baselines (18.91M parameters or smaller, trained on 1–12 digits) fall to **0.0% accuracy** on 18- to 20-digit sums. (Frontier models with chain-of-thought or tools do much better; our claim concerns small models trained from scratch.) We study the **Character-Based Transformer (CBT)**, an 18.95M-parameter character-level model whose tokens carry a two-dimensional (2D) coordinate (p, τ) used by **Decoupled 2D Rotary Position Embeddings (2D-RoPE)**. The coordinates are **not learned**: a deterministic, hand-written parser inside the model's forward pass assigns (p, τ) from token patterns (Section 3.3). Trained on 1–15 digit sums, **CBT-18.95M** answers all 20 of our unseen 16- to 50-digit test additions correctly (20/20; 95% Wilson lower bound ≈84%). On a 26-case tool-routing suite it routes 26/26 correctly (Wilson lower bound ≈87%) with 25/26 exact arguments, using phrasing templates shared with training. On 18 unseen out-of-distribution prompts it routes 16/18 correctly; of those 16, **8 (50.0%) copy the argument exactly** and 11 (68.8%) are exact or near-exact, where near-exact outputs (e.g., `calculator(expr="844 *19")`) would execute incorrectly as tool calls. Controlled ablations at 100k parameters indicate that 2D coordinates account for nearly all of the gain, and that recurrent depth ($R = 3$) gives no measurable benefit for addition (Models 3 and 4 tie at 78.6%); we therefore do not claim recurrence as a contribution for this task. All results use a single training run per configuration and small test sets, and the 1D baselines are not equalized with CBT in training lengths or position augmentation (see Limitations). Because character-level generation is slow at inference, we position CBT as a study of coordinate-aware length generalization at small scale and as a candidate for local tool-routing, not as a replacement for frontier LLMs.

1 Introduction

The prevailing paradigm in contemporary language modeling scales autoregressive Transformers [1, 4] to tens or hundreds of billions of parameters over subword vocabularies generated via Byte-Pair Encoding (BPE). When evaluated on open-ended conversational benchmarks, these models exhibit remarkable fluency and contextual recall. However, it is a well-established empirical finding that small Transformers trained from scratch fail to reliably compute arithmetic additions or symbolic operations on numbers longer than those encountered in their training data [5, 6, 7, 8, 13, 14]. Frontier models with chain-of-thought or tool access do considerably better; our statements concern small models trained from scratch. While such a Transformer can memorize or interpolate addition patterns up to its training ceiling, its accuracy drops precipitously as the number of digits N exceeds the training horizon N_{train}. Prior work has improved length generalization through randomized positional encodings [16], index hints and data formatting [17, 18], Abacus-style embeddings [13], and looped Transformers [19]; we do not compare against tuned versions of these methods (see Limitations).

1.1 Scope, Limitations, and Motivation

Why study in-model addition when a calculator tool exists? Arithmetic here is a testbed for length generalization: it has a compact, exactly checkable algorithm and an unbounded length axis. We do not argue that an in-model adder is a practical substitute for a calculator; the tool-calling results are a separate experiment.

Limitations. (1) The coordinates (p, τ) are hand-designed and assigned by a rule-based parser, not learned; the length-generalization result is therefore conditional on correct coordinate assignment (Section 3.3) and demonstrates column-aligned addition with a fixed-template scratchpad rather than general algorithmic induction. (2) Tool calling is bound to a small set of templates that overlap with training data, the OOD set contains only 18 prompts, near-exact copies are functionally failures, and we do not compare against a fine-tuned small off-the-shelf model. (3) Test sets are small (20 arithmetic cases for 16–50 digits, 26 and 18 for tools, 14 for the 100k ablation), every configuration is a single training run, and we report no seed variance. (4) The 1D baselines are not equalized with CBT: they train on

1–12 rather than 1–15 digits, do not use position-offset augmentation, several are strawmen for arithmetic (e.g., naive BPE without digit splitting), and our Abacus baseline (0.0% in-distribution) reflects an under-tuned implementation rather than the method of McLeish et al. [13]. (5) The only algorithmic task is addition; we do not test tasks where coordinates cannot be assigned by a simple parser (such as multiplication or sorting). (6) Edge-device claims are not backed by CPU or phone latency measurements. (7) Recurrent depth showed no benefit in our addition ablation, and the 1.56M HRM-RoPE model (Table 3) also extrapolates to 20 digits using Decoupled 2D-RoPE alone, confirming that the 2D coordinate mapping—not recurrence or the multi-stage curriculum—is the primary driver of addition extrapolation.

1.2 Narrow Scope of "Algorithmic Generalization" in This Work

Throughout this paper, we use the term **algorithmic generalization** in a narrow, restricted sense—specifically referring to **(1) length extrapolation on column-aligned multi-digit integer sums** given rule-assigned 2D coordinates (training strictly on $1 \leq N \leq 15$ digit additions and evaluating zero-shot on unseen $16 \leq N \leq 50$ digit additions), coupled with **(2) template-guided tool routing and character-level entity copying** on out-of-distribution (OOD) user prompts, in the style of tool-use models [15]. We do not claim general algorithmic induction across arbitrary tasks.

For two positive N-digit integers $A = \sum_{i=0}^{N-1} a_i 10^i$ and $B = \sum_{i=0}^{N-1} b_i 10^i$, column-wise addition is governed by a compact, deterministic finite-state machine (FSM) operating right-to-left from the units column ($i = 0$) to the most significant column ($i = N - 1$):

$$s_i = (a_i + b_i + c_{i-1}) \bmod 10, \quad c_i = \left\lfloor \frac{a_i + b_i + c_{i-1}}{10} \right\rfloor \quad (1)$$

where $a_i, b_i \in \{0, \ldots, 9\}$ are the single-digit operands at column i, $c_i \in \{0, 1\}$ is the binary carry bit emitted to column $i+1$, and $c_{-1} = 0$. Because the entire transition table $\delta : \{0..9\}^2 \times \{0, 1\} \to \{0..9\} \times \{0, 1\}$ consists of only $10 \times 10 \times 2 = 200$ discrete rules, executing addition requires $\mathcal{O}(N)$ identical state transitions regardless of whether $N = 4$, $N = 15$, or $N = 50$.

1.3 From Default Transformer Failures to the Design of CBT

Our research trajectory did not begin with two-dimensional character matrices; it began by testing a **standard default Transformer of decent compact size (18.91M non-embedding parameters, $d_{\text{model}} = 512, L = 6, n_{\text{head}} = 8$)** trained under Chinchilla-style token budgets [3] ($D/N \approx 20 : 1$, up to 400M tokens) on 1- to 12-digit addition problems with explicit step-by-step column scratchpads [9, 10]:

```
47+85=<THINK>(0+7+5)=12(1+4+8)=13</THINK>=132
```

Consistent with prior literature [7, 8], the default 18.91M Transformer achieved 98.4% accuracy on seen 1- to 12-digit lengths and 4.6% on moderate 14- to 16-digit extrapolation, before falling to **0.0% accuracy** on unseen 18- to 20-digit sums.

When the default Transformer failed to generalize, we trained eight alternative architectures under the same compute budget (the eight non-default rows of Table 7): subword BPE with word-level chain-of-thought ($V = 50{,}304$), interleaved philosophy+math co-training, weight-tied Looped Transformers [11, 12], multi-timescale Hierarchical Reasoning Models (HRM-1D), Hybrid Gated Linear Recurrent Units (GLRU-SSM), Harmonic Oscillators (H-NRM), discrete Abacus place embeddings (NoPE) [13], and a compact HRM-Text foundation model. Separate 4k-vocabulary BPE and digit-split BPE variants appear only in Table 6. Several of these 1D baselines achieved 4.6–18.7% on moderate 14–16 digit extrapolation, while **all nine baselines in Table 7 (the default model plus these eight) fell to 0.0% on unseen 18- to 20-digit sums**.

Diagnosing these failures guided the design of our **Character-Based Transformer (CBT)**. Our evidence points to position geometry as the dominant factor, with tokenization secondary. The strongest evidence is within Table 6: a character-level model with 1D-RoPE reaches only 2.3% on unseen 14–20 digit sums, whereas the same character-level tokenization with 2D coordinates reaches 100%. Tokenization still matters: naive 4k BPE scores 0.0% even in-distribution, and digit-split BPE with 2D coordinates reaches only 64.3% on unseen 14–20 digit sums. These comparisons are not a clean 2×2 factor study (training lengths and data differ across rows).

1.3.1 1. Subword Tokens Can Obscure Coordinate Granularity

Subword tokenizers can fuse digit substrings into multi-character tokens. As an *illustrative* (not tokenizer-faithful) example, a BPE-style split of two 8-digit numbers might look like:

$$\texttt{"12345678"} \xrightarrow{\text{BPE}} [\texttt{"12"}, \texttt{"3"}, \texttt{"45"}, \texttt{"67"}, \texttt{"8"}]$$
$$\texttt{"87654321"} \xrightarrow{\text{BPE}} [\texttt{"8"}, \texttt{"7"}, \texttt{"65"}, \texttt{"43"}, \texttt{"21"}]$$

A single token like `"45"` would fuse the 10^3 place (4) and the 10^2 place (5) into one vector at one 1D index, while the ones digit is isolated in `"8"` for the first operand but merged inside `"21"` for the second. Such splits break 1-to-1 place-value alignment between operands.

1.3.2 2. 1D Positional Phase Drift and Repetitive Zero-Loop Generation

Even when tokenized into single characters, a 1D positional encoding (1D-RoPE [2, 14]) assigns a single monotonically increasing index $t = 0, 1, \ldots, T-1$. On unseen 20- to 50-digit numbers ($T \approx 600$ characters at roughly 10 characters per scratchpad column), relative distances $(m - n)$ between an answer column and its operand digits take values never seen in training. We hypothesize that this disperses attention across columns; we did not measure attention entropy for the 1D baselines in the experiments reported here. On repeated zero columns $(10^{19} + 10^{19})$, the 1D baselines in our runs emitted unbroken runs of `(0+0+0)=00` blocks. Informally, once the model generates several identical zero-column blocks in a row, it continues repeating that block until the context limit:

$$P\big(\texttt{"(0+0+0)=00"} \;\big|\; \texttt{"(0+0+0)=00"}^{\times m}\big) \approx 1 \qquad (2)$$

This is a qualitative observation from generation traces, not a derived theorem or a measured probability curve over m.

1.4 The CBT Paradigm and Practical Utility on Low-Resource Devices

We call the model a **Character-Based Transformer (CBT)** because its input units are single characters plus a small set of control tags, with no subword merges; what distinguishes it from a standard character-level Transformer is that each token is positioned by a 2D coordinate rather than a flat index. Weight-tied recurrence is a further design choice that our ablation did not find necessary for addition. As shown in Table 1, CBT generates sequences **character-by-character** (paired with 14 atomic discourse tags like `<THINK>` and `<CALL>`) positioned on a 2D coordinate matrix $(p_t, \tau_t) \in \mathbb{N}^2$. By rotating half of each attention head with the macro row p_t and the other half with the micro column τ_t (**Decoupled 2D-RoPE**), and unrolling $L = 6$ physical layers across $R = 3$ recurrent passes, **CBT-18.95M** answers all 20 unseen 16–50 digit test sums correctly while handling multi-tool dispatch and partial zero-shot entity copying.

Why CBT is Designed for Local Edge Inference Rather Than Trillion-Parameter Scaling: We make an explicit practical claim regarding scale. Scaling CBT as-is to frontier LLM sizes would not make practical sense: because CBT generates text *character-by-character* and evaluates $R = 3$ recurrent passes per character, its autoregressive inference latency is several times slower per word than subword BPE models. CBT's potential value is in **low-resource local inference**. Because it learns intent routing, structured JSON/API tool calling, and exact arithmetic within a tiny 100k to 18.95M parameter footprint, it is a candidate for on-device tool routing. The 36 MB figure is the FP16 weight size only: with $R = 3$, per-token compute is roughly that of a ∼57M-parameter dense model and the KV cache grows with the long character sequences. We report no CPU or phone throughput measurements and no FP16 versus INT8 comparison, so the edge suitability claim remains unverified.

2 Empirical Results

We first present the empirical evaluation of the final converged **CBT-18.95M** checkpoint (`champion_final.pt`, trained across Stage 1 Pretraining, Stage 2 Search Copy SFT, and Stage 3 All-Tools 1-Hop Annealing SFT), followed by tokenization ablations, 1D baseline comparisons, and isolated 100k-parameter component ablations. All benchmark evaluations throughout this paper use deterministic greedy decoding (temperature = 0.0, arg max). Across all tables, we explicitly label the **digit ranges seen during training** ($1 \le N \le 15$ for CBT-18.95M; $1 \le N \le 12$ for baselines) versus **unseen zero-shot extrapolation ranges** ($16 \le N \le 50$).

2.1 Unified Capabilities of CBT-18.95M: Sums, Multi-Tool Routing, and OOD Copying

Tables 2, 3, 4, and 5 present the comprehensive evaluation of our final **CBT-18.95M** model (`champion_final.pt`):

1. **100.0% Zero-Shot Extrapolation from 15 Seen Digits to 50 Unseen Digits (Table 3)**: Trained strictly on numbers of $1 \le N \le 15$ digits, CBT-18.95M answers **20/20 (100.0%; 95% Wilson lower bound ≈84%)** unseen 16–25, 30–40, and 45–50 digit additions, including 50-digit cascading carries ($\underbrace{99\ldots99}_{50} + 1 = 10^{50}$).
2. **Stage 3 All-Tools 1-Hop Alignment (Table 4, 100.0% Routing, 96.2% Exact Args)**: In Stage 2 (`step_7000`), applying 1-hop copy induction only to `search` caused prompts with varied prepositions (like `"What is the weather on Rome"` or `"How many miles is 25 km"`) to over-route to `search`. In Stage 3 (`champion_final.pt`), transitioning *all nine tool schemas* (`get_weather`, `get_time`, `calculator`, `unit_convert`, `database_query`, `fetch_api`, `create_event`, `file_manager`, and `search`) to the entity-agnostic `<THINK>` + 1-hop `<CALL>` pointer architecture achieved **100.0% (26/26) tool routing** and **96.2% (25/26) exact argument extraction**, even routing unseen nonce cities (`"What is the weather in Bloumplik?"` → `get_weather(city="Bloumplik")`). Caveat: Stage

Table 1: How the Character-Based Transformer (CBT) Organizes Characters as a 2D Matrix (p, τ): Unlike 1D indices $t \in \{0, 1, \ldots, T-1\}$ that grow along a flat line, CBT places atomic characters onto a structured 2D coordinate grid. **Top (Arithmetic Mode)**: Macro row p tracks the decimal column rank (10^p) so matching digits in the prompt, scratchpad, and answer share identical p ($\Delta p = 0$), while micro column $\tau \in [0, 9]$ tracks the 10-step column template. **Bottom (Discourse Mode)**: Macro row $p \in \{0..4\}$ tracks the functional discourse block, while micro column τ tracks the character index within that block.

A. Arithmetic 2D Matrix Mode: Prompt `47+85=` followed by Scratchpad `<THINK>(0+7+5)=12(1+4+8)=13</THINK>=132`

Region	**Prompt Operands ($A + B =$)**							**Scratchpad Column 0 (10^0 Ones Place; Col 1 uses $p = 1, \tau = 0..9$)**										**Final Sum**				
Token x_t	4	7	+	8	5	=	<THINK>	(	0	+	7	+	5	)	=	1	2	</THINK>	=	1	3	2
1D Index t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	28	29	30	31	32
Macro p_t (Row)	**1**	**0**	0	**1**	**0**	0	0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	0	0	**1**	**1**	**0**
Micro τ_t (Col)	0	0	0	0	0	0	0	0	1	2	3	4	5	6	7	8	9	0	0	0	0	0

B. Natural Language Discourse 2D Matrix Mode: Multi-Turn Tool-Calling Dialogue (`<USER>` → `<THINK>` → `<CALL>` → `<RESULT>` → `<ANSWER>`)

Discourse Row	**Row $p = 0$: User**					**Row $p = 1$: Thought**		**Row $p = 2$: Tool Call**						**Row $p = 3$: Result**			**Row $p = 4$: Answer**			
Token x_t	<USER>	W	h	a	t...	<THINK>	I	w...		<CALL>	s	e	a	r	c...	<RESULT>	{	"...	<ANSWER>	I	t	...
Macro p_t (Row)	**0**	**0**	**0**	**0**	**0**	**1**	**1**	**1**	**1**	**2**	**2**	**2**	**2**	**2**	**2**	**3**	**3**	**3**	**4**	**4**	**4**	**4**
Micro τ_t (Col)	0	1	2	3	4	0	1	2	3	0	1	2	3	4	5	0	1	2	0	1	2	3

Table 2: Executive Benchmark Summary of Final Converged CBT-18.95M (`champion_final.pt`): Scores are raw accuracies on small test sets with 95% Wilson intervals (no pass/fail threshold is applied). The 16–50 digit tiers contain 20 cases in total (lower bound ≈84%); each model was trained once. The OOD copy rows are computed over the 16 correctly routed prompts of Table 5; exact match is the headline metric.

Capability Domain	**Benchmark Suite / Evaluation Metric**	**Scope / Test Cases**	**Training Exposure**	**Final Score**	**95% Wilson CI**
1. Standalone Sums (2D Math)	Tier 1: In-Distribution Column Addition	1–15 Digits	**Seen in Training (1–15d)**	**100.0%**	–
	Tier 2: Moderate Length Extrapolation	16–25 Digits	**Unseen OOD (+1 to +10d)**	**100.0%**	–
	Tier 3: High Length Extrapolation	30–40 Digits	**Unseen OOD (+15 to +25d)**	**100.0%**	–
	Tier 4: Extreme Length Extrapolation	45–50 Digits	**Unseen OOD (+30 to +35d)**	**100.0%**	–
	Tier 5: 50-Digit Cascading Carry ($99\ldots99+1$)	50 Consecutive Carries	**Unseen OOD (+35d,** 3.3×**)**	**100.0% (20/20)**	[84, 100]%
2. Multi-Tool & Search Suite	Multi-Domain Structured Tool Benchmark	50 Multi-Turn Dialogues	Seen Tool Schemas (6 Domains)	**100.0% (50/50)**	[93, 100]%
	All-Tools 1-Hop Routing Suite (9 Schemas)	26 Phrasing & Edge Cases	Varied Prepositions & Nonces	**100.0% (26/26)**	[87, 100]%
	Multi-Tool Exact Argument & Entity Extraction	26 Phrasing & Edge Cases	1-Hop Pointer Copy from <USER>	**96.2% (25/26)**	[81, 99]%
3. Zero-Shot OOD Queries	OOD Syntactic Tag Completion (`<THINK>...<CALL>`)	18 Unseen Domain Prompts	**100% Unseen Prompts/Domains**	**100.0% (18/18)**	[82, 100]%
	OOD Zero-Shot Tool Selection Accuracy	18 Unseen Domain Prompts	**Unseen Science/Film/Cities/Units**	**88.9% (16/18)**	[67, 97]%
	OOD 1-Hop Character Pointer Copy, **exact match (headline)**	16 Correctly Routed Cases	**Unseen Target Entities**	**50.0% (8/16)**	[28, 72]%
	secondary: exact or near-exact (≥85% chars)	16 Correctly Routed Cases	Unseen Target Entities	68.8% (11/16)	[44, 86]%

3 was trained on the prepositions and templates used in this 26-case suite, whereas Stage 2 never saw them, so the Stage 2 versus Stage 3 comparison is not informative about generalization to new syntax. We did not measure template overlap between the 26 cases and the training generator, and we have no held-out paraphrase set.

3. **Zero-Shot Out-of-Distribution (OOD) Generalization (Table 5)**: On 18 unseen prompts, CBT-18.95M produced well-formed tags for all 18 (18/18) and routed 16/18 (88.9%) to the intended tool. Among the 16 routed cases, **8 (50.0%) copied the argument exactly**; 3 more were near-exact (`"existentialis"`, `"zagnarok"`, `"844 *19"`), giving 11/16 (68.8%) as a secondary, more lenient figure. Because a near-exact string is an incorrect tool argument, exact match is the relevant metric. Exact copies include single words and short lowercase phrases (`"quantum physics"`, `"interstellar"`, `"hyperinflation"`, `"tiramisu"`, `"neutron stars"`, `"Reykjavik"`), but single-word copying is *not* perfect (`"existentialism"`, `"Ragnarok"`, `"theremin"` fail). Multi-word capitalized proper nouns (`"Buenos Aires"` → `"Buris"`, `"Marcus Aurelius"` → `"Arcusnaus"`, `"Kuala Lumpur"` → `"Vuala"`) blend or truncate, plausibly because the synthetic nonce generator trains on single-word pseudowords (a hypothesis we did not test). An embedded digit (`"crispr cas9"`) and a preposition (`"Search for ..."`) each caused a mis-route.

2.2 Isolated Component Ablations of CBT Mechanics (4.24M Token Budget)

Because we tested multiple combinations of CBT components that achieved similar partial extrapolation scores before converging on our final configuration, we conducted a controlled 5-model factor ablation (Table 8) using the exact 100k-parameter budget (`100,672` parameters, $L = 2, d_{\text{model}} = 64, n_{\text{head}} = 4, V = 27$) and the exact same training token volume ($4,236,029$ tokens, 2,000 steps at batch size 32) as the 100k reference model:

1. **1D-RoPE Alone or With Recurrence Fails Completely (Models 1 & 2)**: Under 1D-RoPE, neither $R = 1$ (Model 1, 21.4% overall) nor $R = 3$ recurrence (Model 2, 14.3% overall) solves even a single unseen 14–20 digit problem (0.0% extrapolation; loss stalls above `0.13`).
2. **Decoupled 2D-RoPE Is the Primary Factor (Model 1 vs. Model 3)**: The clean single-factor contrast for positional geometry is Model 1 (1D-

Table 3: Itemized Arithmetic Length Extrapolation Benchmark (1 to 50 Digits), Separating Seen Training Lengths from Unseen Zero-Shot Extrapolation Lengths: Standard 1D-Char and 50k-BPE models (18.91M) and Dual-Loop HRM-RoPE (1.56M) were trained on **1–12 digits** with a 512-token context window; our **CBT-18.95M** was trained on **1–15 digits** with a 1024-token context window. Part II (16–20 digits) is the clean comparison within the 512-token window: HRM-RoPE (1.56M) extrapolates to 20 digits using Decoupled 2D-RoPE without CBT's three-stage curriculum, whereas the 1D models fail. In Part III (25–50 digits), all 512-token models exceed their training context length and are marked `N/A (Context Limit)`.

Digit Length	Training Exposure Status	Representative Test Problem	Expected Output	18.91M 1D-Char	18.91M 50k-BPE	1.56M HRM-RoPE	CBT-18.95M (Ours)
Part I: Seen Training Distribution ($1 \le N \le 12$–15 Digits — Seen During Training)							
2 Digits	Seen in Training	15+25=	40	40 [PASS]	40 [PASS]	40 [PASS]	40 [PASS]
4 Digits	Seen in Training	1234+5678=	6912	6912 [PASS]	6912 [PASS]	6912 [PASS]	6912 [PASS]
8 Digits	Seen in Training	12345678+87654321=	99999999	99999999 [PASS]	99999999 [PASS]	99999999 [PASS]	99999999 [PASS]
8 Digits	Seen in Training (Zeros)	10000000+10000000=	20000000	20000000 [PASS]	20000000 [PASS]	20000000 [PASS]	20000000 [PASS]
12 Digits	Seen in Training	123456789012+876543210987=	999999999999	999999999999 [PASS]	999999999999 [PASS]	999999999999 [PASS]	999999999999 [PASS]
12 Digits	Seen in Training (Overflow)	500000000000+500000000000=	1000000000000	1000000000000 [PASS]	1000000000000 [PASS]	1000000000000 [PASS]	1000000000000 [PASS]
14 Digits	Unseen for 12d / Seen for 15d	12345678901234+876543210987 65=	99999999999999	Zero-Loop [FAIL]	Hallucinates [FAIL]	99999999999999 [PASS]	99999999999999 [PASS]
14 Digits	Unseen for 12d / Seen for 15d	50000000000000+50000000000000=	10^{14} (15 digits)	Zero-Loop [FAIL]	Hallucinates [FAIL]	6×10^{13} [FAIL]	10^{14} [PASS]
15 Digits	Max Seen Bound (CBT-18.95M)	500000000000000+500000000000000=	10^{15} (16 digits)	Zero-Loop [FAIL]	Hallucinates [FAIL]	Untested	10^{15} [PASS]
Part II: Unseen Moderate & Extreme Extrapolation ($16 \le N \le 20$ Digits — Clean Comparison Within 512-Token Context)							
16 Digits	**Unseen OOD (+1d / +4d)**	1111222233334444+8888777766665555=	$16 \times$ '9'	Zero-Loop [FAIL]	Hallucinates [FAIL]	$16 \times$ '9' [PASS]	$16 \times$ '9' [PASS]
18 Digits	**Unseen OOD (+3d / +6d)**	$10^{17} + 10^{17}$ =	2×10^{17}	Loops 000... [FAIL]	Invalid CoT [FAIL]	2×10^{17} [PASS]	2×10^{17} [PASS]
20 Digits	**Unseen OOD (+5d / +8d)**	1234...90 + 8765...09= (20d)	$20 \times$ '9'	Drift [FAIL]	Drift [FAIL]	$20 \times$ '9' [PASS]	$20 \times$ '9' [PASS]
20 Digits	**Unseen OOD (+5d / +8d)**	$10^{19} + 10^{19}$ =	2×10^{19}	Loops 000... [FAIL]	Drift [FAIL]	2×10^{19} [PASS]	2×10^{19} [PASS]
20 Digits	**Unseen OOD (+5d / +8d)**	$\underbrace{999...999}_{20}+1$ =	10^{20} (21 digits)	Loops 000... [FAIL]	Drift [FAIL]	10^{20} [PASS]	10^{20} [PASS]
Part III: Unseen Ultra-Long Horizon Extrapolation ($25 \le N \le 50$ Digits — Exceeds 512-Token Context of 12d Models)							
25 Digits	**Unseen OOD (+10 Extra Digits)**	$\underbrace{999...999}_{25}+1$ =	10^{25} (26 digits)	N/A (Context Limit)	N/A (Context Limit)	N/A (Context Limit)	10^{25} [PASS]
35 Digits	**Unseen OOD (+20 Extra Digits)**	$\underbrace{111...111}_{35}+\underbrace{888...888}_{35}$ =	$35 \times$ '9'	N/A (Context Limit)	N/A (Context Limit)	N/A (Context Limit)	$35 \times$ '9' [PASS]
50 Digits	**Unseen OOD (+35 Extra Digits)**	123...890 + 876...109= (50d)	$50 \times$ '9'	N/A (Context Limit)	N/A (Context Limit)	N/A (Context Limit)	$50 \times$ '9' [PASS]
50 Digits	**Unseen OOD (+35 Extra Digits)**	$\underbrace{999...999}_{50}+1$ =	10^{50} (51 digits)	N/A (Context Limit)	N/A (Context Limit)	N/A (Context Limit)	10^{50} [PASS]

Table 4: Stage 3 All-Tools 1-Hop Induction SFT Alignment (`champion_final.pt`) vs. Stage 2 Search-Only SFT (`step_7000`): Extending the entity-agnostic `<THINK>` + 1-hop `<CALL>` pointer copy architecture across all 9 tool schemas achieves **100.0% (26/26) tool routing** and **96.2% (25/26) exact argument extraction**, resolving preposition and phrasing sensitivities.

Tool Category	User Prompt (`<USER>...`)	Stage 2 Search-Only SFT (`step_7000`)	Stage 3 All-Tools 1-Hop SFT (`champion_final.pt`)	Status
Weather (on)	What is the weather on Rome	search(query="Roe Weather")	<CALL>get_weather(city="Rome")</CALL>	[PASS]
Weather (in)	What is the weather in Rome?	search(query="the weather")	<CALL>get_weather(city="Rome")</CALL>	[PASS]
Weather (short)	Weather in Tokyo	search(query="Tokyo")	<CALL>get_weather(city="Tokyo")</CALL>	[PASS]
Weather (How is)	How is the weather in Paris	search(query="Ineazhather in Paris")	<CALL>get_weather(city="Paris")</CALL>	[PASS]
Weather (Check)	Check the weather for London	search(query="Looath goston")	<CALL>get_weather(city="London")</CALL>	[PASS]
Weather (Unseen Nonce)	What is the weather in Bloumplik?	search(...)	<CALL>get_weather(city="Bloumplik")</CALL>	[PASS]
Time (in EST)	What is the time in EST	search(query="the time")	<CALL>get_time(timezone="EST")</CALL>	[PASS]
Time (What time)	What time is it in JST	search(query="iis STST")	<CALL>get_time(timezone="JST")</CALL>	[PASS]
Unit (Convert)	Convert 10 km to miles	unit_convert(value=10, ...)	<CALL>unit_convert(value=10, from_unit="km", to_unit="miles")</CALL>	[PASS]
Unit (How many)	How many miles is 25 km	calculator(expr="24*1+...")	<CALL>unit_convert(value=25, from_unit="km", to_unit="miles")</CALL>	[PASS]
Search (Real Entity 1)	What are tomatoes	search(query="tomatoes")	<CALL>search(query="tomatoes")</CALL>	[PASS]
Search (Real Entity 2)	Who is Alexander Graham Bell?	search(query="Alexander Graham Bell")	<CALL>search(query="Alexander Graham Bell")</CALL>	[PASS]
Search (Unseen Nonce 1)	What is a bloumplik	search(query="bloumplik")	<CALL>search(query="bloumplik")</CALL>	[PASS]
Search (Unseen Nonce 2)	Define zaimgromp	search(query="zaimgromp")	<CALL>search(query="zaimgromp")</CALL>	[PASS]
Nested JSON (Database)	Query top 5 active records in engineering.	database_query(...)	<CALL>database_query(filter={"department": "engineering", ...})</CALL>	[PASS]
Nested JSON (API)	Fetch latency metrics for EU from /v2/metrics.	fetch_api(...)	<CALL>fetch_api(endpoint="/v2/metrics/latency", params={...})</CALL>	[PASS]
Direct Conversational	Thank you!	<ANSWER>Hello! How can I...</ANSWER>	<ANSWER>You are very welcome! Let me know if you need anything else.</ANSWER>	[PASS]

RoPE, $R = 1, k = 0$) versus Model 3 (Decoupled 2D-RoPE, $R = 1, k = 0$): replacing 1D-RoPE with 2D-RoPE lowers training loss from `0.1864` to `0.0011` and raises accuracy to 100.0% on seen 1–12d and unseen 14–16d sums (78.6%, 11/14 overall).

3. **Recurrence Is Not Needed for Addition in This Ablation (Model 3 vs. Model 4)**: Comparing Model 3 ($R = 1$) against Model 4 ($R = 3$) isolates recurrence: both score 78.6% (11/14) and final loss is slightly worse with $R = 3$ (`0.0017` vs. `0.0011`). We therefore make no claim that recurrent depth contributes to addition at 100k scale. We did not run $R = 1$ versus $R = 3$ at 18.95M, so whether recurrence matters at larger scale or on other tasks is untested.
4. **Shift Masking and Readout Termination (Model 4 vs. Model 5)**: Comparing Model 4 ($k = 0$) and Model 5 ($k = 16$) isolates shift masking on this 14-case set; because they differ on only 1–2 test problems, the comparison is consistent with—but does not statistically establish—a benefit on internal scratchpad columns. At 12.8 epochs (5,000-sample pool), Models 3, 4, and 5 all score 78.6% (11/14) on strict final string match. Inspecting the raw generation tokens of Model 5 (2D-RoPE + $R = 3 + k = 16$) on the 20-digit cascading carry $99\ldots99 + 1$ reveals the following:

`<THINK>(0+9+1)=10(1+9+0)=10...(20 cols)...`
`(1+9+0)=10(1+0+0)=01</THINK>=1`$\underbrace{\texttt{00...00}}_{\text{19 zeros}}$`<EOS>`

Inside `<THINK>...</THINK>`, Model 5's scratchpad was correct across all columns in the 14 test cases. The error is an *off-by-one readout termination* after `</THINK>=`, where the model emits 19 zeros instead of 20 before `<EOS>`. Because the answer-digit coordinates are derived from the scratchpad column count via $p = \max(0, C - 1 - i)$ (Section 3.3), the model itself must decide where the answer ends. Two plau-

Table 5: Zero-Shot Out-of-Distribution (OOD) Generalization of CBT-18.95M (`champion_final.pt`) on 18 Unseen Prompts: 18/18 produced well-formed tags; 16/18 (88.9%) were routed to the intended tool (two drift cases); among the 16 routed cases, **8 (50.0%) copied the argument exactly**, 3 were near-exact (11/16 = 68.8% exact or near-exact), and 5 were partial. Near-exact copies are functionally failures for tool calling: `calculator(expr="844 *19")` computes the wrong product and `search(query="zagnarok")` searches the wrong string. Many prompts reuse phrasing templates seen in training; only the entities are novel.

Unseen OOD Domain	Unseen User Prompt (<USER>...)	Generated Output (<CALL>...</CALL>)	Tool Routing	Character Copy Fidelity
OOD Science	What is quantum physics	<CALL>search(query="quantum physics")</CALL>	[PASS]	[EXACT] (15/15 chars)
OOD Film / Term	What is interstellar	<CALL>search(query="interstellar")</CALL>	[PASS]	[EXACT] (12/12 chars)
OOD Economics	Define hyperinflation	<CALL>search(query="hyperinflation")</CALL>	[PASS]	[EXACT] (14/14 chars)
OOD Cuisine	What is tiramisu?	<CALL>search(query="tiramisu")</CALL>	[PASS]	[EXACT] (8/8 chars)
OOD Astronomy	What are neutron stars	<CALL>search(query="neutron stars")</CALL>	[PASS]	[EXACT] (13/13 chars)
OOD Weather (City)	How is the weather in Reykjavik	<CALL>get_weather(city="Reykjavik")</CALL>	[PASS]	[EXACT] (9/9 chars)
OOD Unit (Inverted)	How many km is 42 miles	<CALL>unit_convert(value=42, from_unit="miles", to_unit="km")</CALL>	[PASS]	[EXACT] (Inverted order parsed)
OOD Timezone	What time is it in GMT	<CALL>get_time(timezone="GMT")</CALL>	[PASS]	[EXACT] (3/3 chars)
OOD Philosophy	What is existentialism?	<CALL>search(query="existentialis")</CALL>	[PASS]	[NEAR-EXACT] (13/14 chars)
OOD Mythology	Tell me about Ragnarok	<CALL>search(query="zagnarok")</CALL>	[PASS]	[NEAR-EXACT] (7/8 chars)
OOD Calculator	Compute 84 * 19	<CALL>calculator(expr="844 *19")</CALL>	[PASS]	[NEAR-EXACT] (6/7 chars)
OOD Deep Learning	What are transformers in deep learning	<CALL>search(query="transwormers of deep")</CALL>	[PASS]	[PARTIAL] (4-word phrase)
OOD Music / Instrument	What is a theremin	<CALL>search(query="ther")</CALL>	[PASS]	[PARTIAL] (prefix copied)
OOD History (2-Word)	Who was Marcus Aurelius?	<CALL>search(query="Arcusnaus")</CALL>	[PASS]	[PARTIAL] (2-word blend)
OOD Weather (2-Word)	What is the weather in Buenos Aires?	<CALL>get_weather(city="Buris")</CALL>	[PASS]	[PARTIAL] (2-word blend)
OOD Weather (2-Word)	Weather on Kuala Lumpur	<CALL>get_weather(city="Vuala")</CALL>	[PASS]	[PARTIAL] (first word copied)
OOD Bio (With Digit)	Explain crispr cas9	<CALL>unit_convert(vanaver="9, ance")</CALL>	[DRIFT]	Digit 9 triggered unit_convert
OOD Architecture	Search for brutalist architecture	<CALL>get_weather(city="Oncordar")</CALL>	[DRIFT]	Preposition for triggered get_weather

Table 6: The Tokenization and Positional Geometry Matrix: Comparing our 2D Matrix Character + Control-Token CBT formulation against 1D Character models, Subword BPE models (4k and 50k vocabularies), Digit-Split 4k BPE, and Learned 2D positional tables, explicitly contrasting **Seen Training Digits** vs. **Unseen Extrapolation Digits**. Vocabulary sizes differ by variant: 115 is the full CBT-18.95M tokenizer, 111 the CBT-200k and Learned-2D variants, and 27 the digit-and-symbol character models. All rows showing '/50' in the Tool Calling column were evaluated on the identical 50-dialogue structured tool suite (6 domains), whereas CBT-18.95M was evaluated on the 26-case 9-schema 1-hop suite of Table 4. Rows were trained with different digit ranges and data and are not a controlled 2×2 study. The unseen-sum column pools 14–20 digits, whereas Table 7 reports 14–16 digits separately (hence 2.3% here and 4.6% there for the 1D character model).

Tokenizer Granularity	Positional Geometry	Vocab (V)	Trained Digits	Tool Calling	Seen Sums (In-Dist)	Unseen Sums (14–20d)	Unseen Cascade Carry
Subword BPE (50k)	1D-RoPE (Flat Line)	50,304	1–12 Digits	N/A (Word-CoT)	99.1% (1–12d)	**0.0%** (Unseen 18–20d)	**0.0%** (Unseen 20d)
Subword BPE (Naive 4k)	1D / 2D-RoPE	4,000	1–12 Digits	**100.0% (50/50)**	**0.0%** (1–12d)	**0.0%** (Unseen 14–20d)	**0.0%** (Unseen 20d)
Digit-Split BPE (4k)	2D-RoPE Matrix	4,114	1–12 Digits	**100.0% (50/50)**	83.3% (1–12d)	64.3% (Unseen 14–20d)	**0.0%** (Unseen 20d)
Atomic Character	1D-RoPE (Flat Line)	27 / 115	1–12 Digits	**100.0% (50/50)**	98.4% (1–12d)	2.3% (Unseen 14–20d)	**0.0%** (Unseen 20d)
Atomic Character	NoPE + Abacus Place	27	1–12 Digits	N/A	**0.0%** (1–12d)	**0.0%** (Unseen 14–20d)	**0.0%** (Unseen 20d)
Atomic Character	Learned 2D Lookup Table	111	1–12 Digits	98.0% (49/50)	100.0% (1–12d)	**0.0%** (Capped $p > 12$)	**0.0%** (Unseen 20d)
CBT-200k (Char + Control)	**2D-RoPE Matrix** ($R = 3$)	**111**	**1–12 Digits**	**100.0% (50/50)**	**100.0% (1–12d)**	**100.0% (Unseen 14–20d)**	**100.0% (Unseen 20d)**
CBT-18.95M (Char + Control)	**2D-RoPE Matrix** ($R = 3$)	**115**	**1–15 Digits**	**100.0% (26/26)**	**100.0% (1–15d)**	**100.0% (Unseen 16–50d)**	**100.0% (Unseen 50d)**

sible (untested) hypotheses for this off-by-one error are: (a) the clamp $p = \max(0, C-1-i)$ assigns $p = 0$ to both the true units digit ($i = C-1$) and any extra overflow digit ($i \geq C$), making the last answer digit and the post-answer position share the same macro coordinate $p = 0$; and (b) after shift masking moves $p > 0$ by $\Delta \in [0, 16]$ while $p = 0$ stays fixed, the $p = 1 \to p = 0$ boundary is under-trained at 12.8 epochs. When the exact same architecture is trained for 64 epochs on a 1,000-sample pool (`CBT-100k Ref`, 4.24M tokens) or scaled to **CBT-18.95M** (247.3M tokens), the readout error disappeared in our tests (14/14 for `CBT-100k Ref`; 20/20 at 16–50 digits for CBT-18.95M), though repeating a 1,000-sample pool 64 times raises the possibility of overfitting to training length distributions rather than purely resolving boundary attention.

3 Architecture, 2D Matrix Embeddings, Tokenizer, and Training Data

In this section, we provide a complete technical specification of **CBT-18.95M**, detailing its hybrid character-control tokenizer, Decoupled 2D-RoPE embeddings, weight-tied recurrent depth, and the exact composition of the training datasets.

3.1 Atomic Character and Structural Control Tokenizer ($V = 115$)

To eliminate BPE subword chunking distortions while retaining clear structural boundaries for multi-turn agentic dialogues, our tokenizer (`ScaledTokenizer`) defines a compact vocabulary $\mathcal{V}$ of size $V = 115$ for CBT-18.95M (the smaller CBT-200k and 100k variants use character subsets with $V = 111$ and $V = 27$ respectively), partitioned into two disjoint subsets:

$$\mathcal{V} = \mathcal{V}_{\text{ctrl}} \cup \mathcal{V}_{\text{char}}, \quad |\mathcal{V}_{\text{ctrl}}| = 14, \quad |\mathcal{V}_{\text{char}}| = 101 \tag{3}$$

Table 7: Cross-Architecture Comparison on Seen vs. Unseen Digit Lengths: Nine 1D/NoPE baselines (the default character model plus eight alternatives, trained on 1–12 digits; several achieve 4.6–18.7% on moderate 14–16 digit extrapolation, while all nine fall to 0.0% on unseen 18–20 digit problems) compared against the 2D-RoPE Character-Based Transformer (CBT) family. CBT-18.95M trained on 1–15 digits and the baselines on 1–12 digits; baselines used no position-offset augmentation and no tuned index hints, so this is not an equalized comparison. The MicroMath/Abacus row (0.0% even in-distribution) likely reflects an under-tuned implementation rather than a property of Abacus embeddings [13].

Architecture Paradigm	Non-Emb Params	Positional Encoding	Trained Digits	Seen In-Dist	Unseen Mod OOD	Unseen Ext OOD	Unseen Extrapolation Summary
Standard Char Transformer	18.91M	1D-RoPE	1–12 Digits	98.4%	4.6% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Subword BPE Word-CoT	18.91M	1D-RoPE	1–12 Digits	99.1%	8.0% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Philo-Math Co-Trained GPT	18.91M	1D-RoPE	1–12 Digits	94.2%	5.3% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Looped Recurrent GPT ($K = 6$)	18.91M	1D-RoPE	1–12 Digits	100.0%	12.0% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Hierarchical HRM-1D	18.91M	1D-RoPE	1–12 Digits	100.0%	18.7% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Hybrid GLRU-SSM Transformer	18.91M	1D-RoPE + Linear State	1–12 Digits	100.0%	10.7% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
Harmonic ALU (H-NRM)	18.91M	Poly-Freq Oscillators	1–12 Digits	97.8%	8.0% (14–16d)	0.0% (18–20d)	No length beyond 12d reliably
MicroMath (Abacus NoPE, 1.36M)	1.36M	Discrete Place Only (p)	1–12 Digits	0.0%	0.0% (14–16d)	0.0% (18–20d)	0 Digits (Under-tuned / Collapse)
Foundation HRM-Text (219k tok/s)	1.44M	1D-RoPE + Additive Loop	1–12 Digits	66.7%	0.0% (14–16d)	0.0% (18–20d)	No length beyond 8d reliably
HRM-RoPE (Dual-Loop)	1.56M	**Decoupled 2D-RoPE**	**1–12 Digits**	100.0%	66.7% (14–16d)	**100.0% (18–20d)**	**20 Digits (+8 Unseen Digits)**
CBT-100k ($L = 2, R = 3$)	98.9k	**Decoupled 2D-RoPE**	**1–12 Digits**	**100.0%**	**100.0% (14–16d)**	**100.0% (18–20d)**	**20 Digits (+8 Unseen Digits)**
CBT-200k ($L = 4, R = 3$)	197.8k	**Decoupled 2D-RoPE**	**1–12 Digits**	**100.0%**	**100.0% (14–16d)**	**100.0% (18–20d)**	**20 Digits (+8 Unseen) + Tools**
CBT-18.95M ($L = 6, R = 3$)	**18.89M**	**Decoupled 2D-RoPE**	**1–15 Digits**	**100.0%**	**100.0% (16–25d)**	**100.0% (30–50d)**	**50 Digits (+35 Unseen) + Tools**

Table 8: Isolated Component Ablation Study of CBT Mechanics at 100k Scale (100,672 Total Params, Identical 4.24M Token Budget): All 6 models share the exact same 100k character-level backbone ($L = 2, d_{\text{model}} = 64, n_{\text{head}} = 4, V = 27$) and vary three factors indicated in the top header rows: **Positional Encoding** (`1D-RoPE` vs. `Decoupled 2D-RoPE`), **Recurrent Depth** ($R = 1$ single pass vs. $R = 3$ weight-tied passes), and **Coordinate Shift Masking** ($k = 0$ off vs. $k = 16$ random column shift). Models 1–5 are trained for 2,000 steps on a 5,000-sample pool (≈ 12.8 epochs); the 6th column (`CBT-100k Ref`) trains the Model 5 configuration for 64 epochs on a 1,000-sample pool (`dense_100k_2d_rec3_full.pt`). In Model 5, the internal `<THINK>` scratchpad columns were exact in all 14 cases, with the 3 readout misses attributable to an off-by-1 trailing zero before `<EOS>`. Because the 64-epoch reference repeats a 1,000-sample pool 64 times, we cannot rule out memorization or overfitting to boundary patterns on that checkpoint. The 100k ablation uses $k = 16$, whereas CBT-18.95M uses $k = 32$ (Eq. 6). With 14 test cases (where 1–3 cases separate models), Models 3, 4, and 5 cannot be statistically distinguished, and each configuration was trained once.

Benchmark Tier / Configuration	Training Exposure	Model 1 *1D Baseline*	Model 2 *1D + Recurrence*	Model 3 *2D-RoPE Only*	Model 4 *2D + Recurrence*	Model 5 *2D + Rec + Shift*	CBT-100k Ref *2D + Rec + Shift (64 Ep)*
Positional Geometry	—	1D-RoPE (Flat)	1D-RoPE (Flat)	**Decoupled 2D-RoPE**	**Decoupled 2D-RoPE**	**Decoupled 2D-RoPE**	**Decoupled 2D-RoPE**
Recurrent Passes (R)	—	$R = 1$ (2 Layers)	$R = 3$ (6 Eff. Layers)	$R = 1$ (2 Layers)	$R = 3$ (6 Eff. Layers)	$R = 3$ (6 Eff. Layers)	$R = 3$ (6 Eff. Layers)
Random Shift Masking (k)	—	Off ($k = 0$)	Off ($k = 0$)	Off ($k = 0$)	Off ($k = 0$)	**On** ($k = 16$)	**On** ($k = 16$)
Training Pool / Epochs	4.24M Tokens	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	5k Pool (12.8 Ep)	1k Pool (64.0 Ep)
Final Pretraining Loss	4.24M Tokens	0.1864	0.1353	0.0011	0.0017	0.0017	0.0016
Seen In-Dist (1–12 Digits)	**Seen (1–12d)**	50.0% (3/6)	33.3% (2/6)	**100.0% (6/6)**	**100.0% (6/6)**	**100.0% (6/6)**	**100.0% (6/6)**
Unseen Mod Extrap (14–16d)	**Unseen (+2 to +4d)**	0.0% (0/3)	0.0% (0/3)	**100.0% (3/3)**	**100.0% (3/3)**	**100.0% (3/3)**	**100.0% (3/3)**
Unseen Ext Extrap (18–20d)	**Unseen (+6 to +8d)**	0.0% (0/3)	0.0% (0/3)	33.3% (1/3)	33.3% (1/3)	**66.7% (2/3)**	**100.0% (3/3)**
$10^{17} + 10^{17} =$ (18d Zeros)	Unseen (+6d)	[FAIL]	[FAIL]	[FAIL]	[FAIL]	2e17 [PASS]	2e17 [PASS]
1234...90+8765...09= (20d)	Unseen (+8d)	[FAIL]	[FAIL]	99...99 [PASS]	99...99 [PASS]	99...99 [PASS]	99...99 [PASS]
5555...55+4444...45= (20d)	Unseen (+8d)	[FAIL]	[FAIL]	[FAIL]	[FAIL]	*100% Think / Off-by-1 0*	1e20 [PASS]
Unseen 20d Zeros & Carry	**Unseen (+8d)**	0.0% (0/2)	0.0% (0/2)	50.0% (1/2)	50.0% (1/2)	*100% Think / Off-by-1 0*	**100.0% (2/2)**
$10^{19} + 10^{19} =$ (20d Zeros)	Unseen (+8d)	[FAIL]	[FAIL]	2e19 [PASS]	2e19 [PASS]	*19/19 Cols OK, 18 0s out*	2e19 [PASS]
$99\ldots99 + 1 =$ (20d Carry)	Unseen (+8d)	[FAIL]	[FAIL]	[FAIL]	[FAIL]	*21/21 Cols OK, 19 0s out*	1e20 [PASS]
Internal `<THINK>` Math Acc	All 14 Cases	21.4% (3/14)	14.3% (2/14)	78.6% (11/14)	78.6% (11/14)	**100.0% (14/14 Exact)**	**100.0% (14/14)**
Strict Final String Match	All 14 Cases	21.4% (3/14)	14.3% (2/14)	78.6% (11/14)	78.6% (11/14)	78.6% (11/14)	**100.0% (14/14)**

1. **Structural Control Tokens ($\mathcal{V}_{\text{ctrl}}$, IDs $0 \ldots 13$)**: Consists of 4 standard sequence control tokens (`<PAD>`, `<BOS>`, `<EOS>`, `<UNK>`) and 10 paired discourse state delimiters that structure agentic reasoning:

   `<USER>`, `</USER>`, `<THINK>`, `</THINK>`, `<CALL>`, `</CALL>`, `<RESULT>`, `</RESULT>`, `<ANSWER>`, `</ANSWER>`

2. **Atomic ASCII Characters ($\mathcal{V}_{\text{char}}$, IDs $14 \ldots 114$)**: Contains all printable ASCII characters—lowercase letters (`a--z`), uppercase letters (`A--Z`), single digits (`0--9`), punctuation symbols (`+`, `=`, `(`, `)`, `{`, `}`, `"`, etc.), space, newline, and tab.

During tokenization, a greedy left-to-right prefix scanner checks if the input substring at index i matches any tag in $\mathcal{V}_{\text{ctrl}}$. If matched, the entire tag (e.g., `<THINK>`) is emitted as a single token ID; otherwise, the single character $s[i]$ is emitted as its atomic character ID.

3.2 Decoupled 2D Rotary Position Embeddings (2D-RoPE)

In **Decoupled 2D-RoPE**, every token x_t is associated with a 2D coordinate pair $(p_t, \tau_t) \in \mathbb{N}^2$, where p_t is the **Macro (Row) Coordinate** and τ_t is the **Micro (Column) Coordinate**. Within each attention head of dimension $d_k = 64$, we split the query and key vectors into two orthogonal 32-dimensional subspaces:

$$\tilde{\mathbf{q}}_t^{(h)} = \begin{bmatrix} \mathcal{R}_\Theta(p_t)\, \mathbf{q}_{t,:32}^{(h)} \\ \mathcal{R}_\Theta(\tau_t)\, \mathbf{q}_{t,32:}^{(h)} \end{bmatrix}, \quad \tilde{\mathbf{k}}_t^{(h)} = \begin{bmatrix} \mathcal{R}_\Theta(p_t)\, \mathbf{k}_{t,:32}^{(h)} \\ \mathcal{R}_\Theta(\tau_t)\, \mathbf{k}_{t,32:}^{(h)} \end{bmatrix} \tag{4}$$

For any coordinate scalar $c \in \{p_t, \tau_t\}$ and a 32-dimensional subspace vector $\mathbf{u} \in \mathbb{R}^{32}$, $\mathcal{R}_\Theta(c)\mathbf{u}$ applies 2×2 planar rotations across the 16 channel pairs $(2j, 2j+1)$ for $j \in \{0, 1, \ldots, 15\}$ with $\theta_j = 10000^{-2j/32}$.

Proposition 1 (Zero Phase Drift & Scale-Invariant Col-

Table 9: Exact Layer-by-Layer Parameter Topology of CBT-18.95M: $d_{\text{model}} = 512$, $n_{\text{head}} = 8$ ($d_k = 64$), $L = 6$ physical blocks, $R = 3$ recurrent passes ($L_{\text{eff}} = 18$), bias disabled, tied input/output embeddings.

Component / Module	Tensor Shape	Parameters	% of Total
Token Embedding (W_{te})	115×512	58,880	0.31%
Per Physical Block ($l \in \{1..6\}$):			
Pre-Attn LayerNorm (`ln_1`)	2×512	1,024	0.005%
Fused QKV Proj (`c_attn`)	512×1536	786,432	4.15%
Attention Output Proj (`c_proj`)	512×512	262,144	1.38%
Pre-MLP LayerNorm (`ln_2`)	2×512	1,024	0.005%
MLP Up-Proj (`c_fc`, 4×)	512×2048	1,048,576	5.54%
MLP Down-Proj (`mlp.c_proj`)	2048×512	1,048,576	5.54%
Subtotal per Physical Block	—	*3,147,776*	*16.62%*
All 6 Physical Blocks ($L = 6$)	$6 \times 3,147,776$	18,886,656	99.69%
Recurrent Unrolling ($R = 3$)	Weight-Tied ($L_{\text{eff}} = 18$)	+0	0.00%
Decoupled 2D-RoPE Cache	Non-Parametric (cos, sin)	+0	0.00%
Final LayerNorm (`ln_f`)	2×512	1,024	0.005%
LM Output Head ($W_{\text{lm_head}}$)	Tied to W_{te}	+0	0.00%
Non-Embedding Parameters	—	**18,887,680**	**99.69%**
Total Model Parameters	—	**18,946,560**	**100.00%**

umn Alignment). *Let q_t be a scratchpad query token in decimal column p and k_j be the corresponding operand digit in column p. Under Decoupled 2D-RoPE (Eq. 4), the pre-softmax attention logit $\langle \tilde{\mathbf{q}}_t^{(h)}, \tilde{\mathbf{k}}_j^{(h)} \rangle / \sqrt{d_k}$ is strictly independent of the total number of digits N:*

$$\left(\tilde{\mathbf{q}}_t^{(h)}\right)^\top \tilde{\mathbf{k}}_j^{(h)} = \left(\mathbf{q}_{t,:32}^{(h)}\right)^\top \mathbf{k}_{j,:32}^{(h)} + \left(\mathbf{q}_{t,32:}^{(h)}\right)^\top \mathcal{R}_\Theta(\tau_t)\, \mathbf{k}_{j,32:}^{(h)} \tag{5}$$

Proof. Because both the scratchpad token and the prompt operand digit belong to decimal column 10^p, $p_t = p_j = p \implies \Delta p = 0 \implies \mathcal{R}_\Theta(0) = \mathbf{I}_{32}$, while $\tau_t \in \{0, \ldots, 9\}$ is strictly bounded for all $N \in \mathbb{N}$. □

The proposition is conditional on the coordinates being assigned correctly: it states that a single logit between a scratchpad query token and its corresponding operand digit does not depend on N, not that the model computes carries correctly or that the coordinates are produced by the model. Moreover, Proposition 1 covers only scratchpad-to-operand attention; for the final answer digits after `</THINK>=`, the parser sets $p = \max(0, C - 1 - i)$ using the model's emitted scratchpad column count C, so answer-to-scratchpad alignment depends on the model having emitted the right number of scratchpad columns. Since RoPE depends only on relative offsets, the reason shift masking is considered during training is not absolute position but the large offset between scratchpad rows ($p > 0$) and the prompt tokens at $p = 0$ that appears at test length N but was never seen at training length.

3.3 Coordinate Assignment at Inference

The coordinates are computed by a deterministic, hand-written rule-based parser (`compute_2d_coords` in the `scaled_t4` model file) that is called inside the forward pass over the token ids; it is applied identically in training and in autoregressive generation, and the model does not predict (p, τ). The rules are:

- *Operands*: a digit in the first (second) operand gets p equal to its distance from the `+` (`=`) token minus one, so units digits have $p = 0$; $\tau = 0$.
- *Scratchpad*: inside `<THINK>`, each `(` token starts a new column (p increments, τ resets to 0), and τ counts tokens since the `(`, capped at 9. The model therefore decides when to advance a column by emitting `(`; the parser does not count 10-character templates. Because column advancement is keyed on `(`, the model can advance p only by emitting `(`, which also means a single spurious or missing `(` would shift all subsequent column coordinates. In our 2D-RoPE test runs (Models 3–5, `CBT-100k Ref`, and `CBT-18.95M`), no test error arose from a mis-emitted `(` inside `<THINK>`; the 1D baselines, by contrast, failed by repeating valid 10-character `(0+0+0)=00` blocks too many times rather than corrupting the `(` delimiter syntax.
- *Answer*: after `</THINK>=`, the i-th digit emitted gets $p = \max(0, C-1-i)$ where C is the number of scratchpad columns. The answer length is not given to the model; extra digits are clamped at $p = 0$, so the unit digit ($i = C - 1$) and any extra overflow digit ($i \geq C$) receive the same coordinate $p = 0$. The model must therefore identify the termination point itself, and this $p = 0$ clamp—alongside the $p = 1 \to p = 0$ shift-masking boundary—is a plausible cause of the off-by-one readout error in Table 8.
- *Discourse mode*: if `<USER>` is present, $p \in \{0, \ldots, 4\}$ is the index of the most recent segment tag (`<USER>`, `<THINK>`, `<CALL>`, `<RESULT>`, `<ANSWER>`) and τ counts tokens since that tag.

Thus the operand-to-column alignment (the operand and column ranks) is supplied by the parser, and the length-generalization result should be read as conditional on this hand-designed alignment. The parser also assumes a fixed prompt format (`A+B=`), and is a Python loop over the batch that is not optimized for speed.

During training, we apply **Selective Coordinate Shift Masking**:

$$p_t \leftarrow p_t + \Delta \cdot \mathbb{I}(p_t > 0), \quad \Delta \sim \mathcal{U}\{0, 1, \ldots, k\} \tag{6}$$

with $k = 32$ for CBT-18.95M and $k = 16$ for the 100k ablation models of Table 8. This exposes $p > 0$ arithmetic columns to indices up to $p = 47$ during 15-digit training (with $k = 32$), while keeping the `<USER>` prompt ($p = 0$) anchored at the origin. It is closely related to randomized positional encodings [16].

3.4 Weight-Tied Recurrent Depth with $1/\sqrt{R}$ Scaling

We unroll the $L = 6$ physical Transformer blocks across $R = 3$ recurrent passes per forward step ($L_{\text{eff}} = 18$ layers at zero parameter overhead) using a $1/\sqrt{R}$-scaled residual update:

$$\mathbf{x} \leftarrow \mathbf{x} + \frac{1}{\sqrt{R}} \left(\text{Block}_l(\mathbf{x}, p, \tau) - \mathbf{x}\right) \tag{7}$$

The $1/\sqrt{R}$ factor is a heuristic residual damping choice. We do not derive a variance bound for it, and our ablation (Table 8) shows no benefit from recurrence for addition. We therefore drop the variance comparison between $1/\sqrt{R}$ scaling and unscaled accumulation that appeared in earlier drafts.

3.5 Three-Stage Training Data & 1-Hop Induction Curriculum

Table 10 summarizes the three-stage training protocol that produced `champion_final.pt` on dual NVIDIA T4 GPUs (FP16 AMP):

1. **Stage 1 (Joint Pretraining, 45,059 steps, 247.3M tokens, Loss `0.000094`)**: Trained on a 50%/50% mixture of 1–15 digit column addition (40% asymmetric lengths, 25% symmetric lengths, 20% cascading carries $99\ldots9 + b$, 15% power-of-ten zero chains) and 6-category structured tool dialogues, with -100 loss masking on prompts and `<RESULT>` payloads.
2. **Stage 2 (Search Copy Induction SFT, 7,000 steps, Loss `0.004834`)**: In early SFT tests, repeating an unfamiliar entity X inside `<THINK>` before calling `<CALL>` caused a *2-Hop Copy Bottleneck*, where character n-gram priors hallucinated phonotactic nonces (`"tomatoes"` → `"smoom"`, `"bloumplik"` → `"cheat"`). Decoupling `<THINK>` into an invariant intent statement and training on 65% procedurally generated phonotactic pseudowords ($L_{\text{memorize}} \to \infty$) forced a **1-hop induction pointer head** ($p = 2 \to p = 0$) to copy unseen strings character-by-character.
3. **Stage 3 (All-Tools 1-Hop Annealing SFT, 4,000 steps, LR $5 \times 10^{-5} \to 5 \times 10^{-6}$, Loss `0.007532`)**: Extended the 1-hop pointer copy formulation across all 9 tool schemas (`get_weather`, `get_time`, `calculator`, `unit_convert`, `database_query`, `fetch_api`, `create_event`, `file_manager`, and `search`) alongside a 10% 1–15 digit math replay buffer, achieving **100.0% (26/26) tool routing**, **96.2% (25/26) exact argument extraction**, and **100.0% 1–50 digit sum extrapolation**.

3.6 Hyperparameters and Release

Defaults in the training code (`train_and_eval.py` and `sft_search.py` under `champion_model/scaled_t4/`): AdamW with weight decay 0.01, peak learning rate 10^{-3} with cosine decay, gradient-norm clipping at 1.0, per-GPU batch size 32 on 2 GPUs, context length (block size) 1024 tokens with up to 4096 coordinate indices, $k = 32$ shift range, $R = 3$, FP16 autocast with gradient scaling. Stage 3 SFT uses a learning rate from 5×10^{-5} to 5×10^{-6}. Context length is not the limiting factor at 50 digits (about 10 characters per scratchpad column gives roughly 600 tokens), but the 1D baselines were trained with a 512-token block, so their 20+ digit failures are partly confounded with context length; their failures at 14–16 digits are not. Arithmetic and tool data are produced by procedural generators in the repository (`src/arithmetic.py` and the dataset modules); the code, data generators, and checkpoints are available in the project repository.

4 Discussion, Limitations, and Future Horizons

4.1 Positioning Within Prior Length-Generalization Work

Prior studies have shown that standard Transformers fail to extrapolate arithmetic beyond training lengths under 1D positional encodings [5, 7, 8, 14]. Several lines of work address this by modifying positional information or data formatting: Shen et al. [6], Zhou et al. [17], and Duan & Shi [18] use explicit index hints and structured scratchpads; Ruoss et al. [16] randomize positional indices across a wider range during training (closely related to our shift masking in Eq. 6); McLeish et al. [13] introduce Abacus embeddings that align digits of equal place value; and Fan et al. [19] use looped Transformers for iterative algorithms. Our Decoupled 2D-RoPE is philosophically similar to Abacus embeddings and explicit index hints in that it supplies place-value alignment (p, τ) via a rule-based parser rather than learning it from raw text, while extending the coordinate grid to a two-dimensional (p, τ) RoPE subspace that accommodates both multi-character column scratchpads and multi-segment tool dialogues.

4.2 Why CBT Suits Local Edge Inference Rather Than Brute-Force Scaling

A central takeaway of our work is that **CBT is not meant to replace subword tokenization in massive trillion-parameter cloud LLMs as-is**. Because CBT generates text *character-by-character* and executes

Table 10: Complete Three-Stage Training Data Composition, Digit Horizon Boundaries, and Loss Masking Specification for CBT-18.95M (`champion_final.pt`): Across all stages, arithmetic training is strictly bounded to **1–15 digits** ($N \leq 15$). All 16- to 50-digit evaluations are 100% zero-shot out-of-distribution extrapolations.

Training Stage	Dataset Component / Sub-Distribution	Pattern / Schema Specification	Trained Digit / Length Scope	Mixture Weight	Supervised Target (Masking Rule)
Stage 1: Pretraining	**A. Arithmetic State-Machine Corpus (50%)**	$A + B =$ <THINK>$(c + a + b) = c's \ldots$</THINK>$= S$	**Strictly 1–15 Digits** ($N \leq 15$)	**50.0% of Pretrain**	Prompt $A + B$ = masked (-100)
	1. Asymmetric & Symmetric Random Sums	$d_A, d_B \sim \mathcal{U}[1, 15]$ (40% asymmetric, 25% symmetric)	$1 \leq d_A, d_B \leq 15$ (0% $>$ 15d)	32.5% (65% of Math)	Scratchpad + Final Sum supervised
	2. Cascading Carry & Zero Chains	$99 \ldots 9 + b$ (20%) and $k_1 10^{d_1 - 1} + k_2 10^{d_2 - 1}$ (15%)	$2 \leq d \leq 15$ (0% $>$ 15d)	17.5% (35% of Math)	Scratchpad + Final Sum supervised
	B. Multi-Domain Tool Corpus (50%)	<USER> → <THINK> → <CALL> → <RESULT> → <ANSWER>	$T \leq 512$ Characters	**50.0% of Pretrain**	<USER> & <RESULT> masked (-100)
	Single-Tool, Chaining, JSON, Error, Chat	6 interaction categories across cities, timezones, units	1–3 Conversational Turns	50.0% (6 Equal Cats)	<THINK>, <CALL>, <ANSWER>
Stage 2: Search SFT	**Open-Domain Search 1-Hop Copy (7k steps)**	Entity-agnostic <THINK> + `search(query="X")`	65% Dynamic Nonces / 35% Real	**81.25% (26/32 Batch)**	Forces $L_{\text{memorize}} \to \infty$ pointer copy
	Math & Multi-Tool Replay	1–15d carry chains + tool schemas	**Strictly 1–15 Digits** ($N \leq 15$)	**18.75% (6/32 Batch)**	Preserves 50d math & tool syntax
Stage 3: All-Tools SFT	**Universal 9-Tool 1-Hop Annealing (4k steps)**	All 9 tools (`weather, time, unit, calc, db, api, search...`)	Varied Prepositions & Nonces	**90.0% of Stage 3**	Resolves preposition/routing drift
	High-Digit Arithmetic Carry Replay	Cascading carries & random sums ($N \leq 15$d)	**Strictly 1–15 Digits** ($N \leq 15$)	**10.0% of Stage 3**	Maintains 100.0% 1–50d math

$R = 3$ recurrent passes per character, generating a 50-word paragraph requires ~ 300 sequential forward passes (compared to ~ 65 subword passes in a BPE model). At billion-parameter scale, this character-by-character serial bottleneck would make inference prohibitively slow.

However, at compact scales (**100k to 18.95M parameters**), individual forward passes involve a small matrix footprint (36 MB of FP16 weights, though $R = 3$ recurrence triples the per-token FLOPs to roughly that of a ∼57M-parameter dense model and character-level sequences expand the KV cache). We did not benchmark latency or throughput on consumer CPUs or mobile hardware, nor did we compare against fine-tuned small off-the-shelf subword models on tool calling. Subject to those unverified practical constraints, CBT's ability to handle intent classification, 9-schema tool routing (26/26 on template-matched prompts; 8/16 exact argument copying on OOD prompts), and 50-digit sums within a 36 MB weight file suggests potential utility for:

- **Low-Resource Local Tool Routers**: On-device assistants on mobile phones, embedded IoT controllers, or local desktop daemons that parse user commands into structured API/JSON calls (`get_weather`, `file_manager`, `database_query`) without cloud connectivity.
- **Deterministic Symbolic Co-Processors**: Compact local modules that execute exact character-level formatting, arithmetic, and pointer extraction without subword hallucination.

4.3 Open Questions and Further Testing

While CBT-18.95M shows that placing characters on a hand-assigned 2D coordinate grid (p, τ) yields correct sums on our unseen 16–50 digit test cases and works for small-scale tool calling, our OOD and ablation experiments leave several questions open:

1. **Sensitivity to Epoch Reinforcement and Boundary Coordinates**: As shown in our 100k component ablations (Table 8), while several combinations of 2D-RoPE and recurrence achieved identical 78.6% strict match at 12.8 epochs, the off-by-one trailing zero readout error at the $p = 1 \to p = 0$ boundary disappeared in our runs with higher epoch density (64 epochs on a 1,000-sample pool) or larger capacity (18.95M); however, these single-run comparisons do not isolate whether the fix came from boundary exposure, memorization of the small pool, or model scale.
2. **Multi-Word OOD Pointer Spans and General Algorithms**: In zero-shot OOD testing (Table 5), two-word capitalized proper nouns (`"Buenos Aires"` → `"Buris"`) still experience partial compression, and our algorithmic generalization scope here focused on $\mathcal{O}(N)$ column sums rather than $\mathcal{O}(N^2)$ multiplication or arbitrary graph algorithms.
3. **Learned Coordinates and Other Tasks**: Emitting hundreds of scratchpad characters is an inefficient simulation of an ALU, and the coordinates are hand-assigned. Important next steps are learning or inferring (p, τ), testing on a second task whose coordinates are not trivial (multiplication, sorting, string reversal), testing 500+ random problems at every length from 16 to 100 digits with multiple seeds, and comparing against equalized 1D baselines with randomized positions, index hints, and tuned Abacus embeddings [16, 17, 18, 19].

5 Conclusion

Starting from the failure of a standard 18.91M-parameter Transformer and eight architectural baselines on unseen 18- to 20-digit sums (**0.0% extrapolation at 18–20 digits**, despite 4.6–18.7% on moderate 14–16 digit sums), our experiments suggest that position geometry is the dominant factor (character-level 1D-RoPE: 2.3% across 14–20d; character-level 2D: 100%), with subword tokenization secondary. We studied the **Character-Based Transformer (CBT)**, which generates atomic characters and structural control tags on a two-dimensional coordinate matrix (p, τ) via **Decoupled 2D-RoPE** and weight-tied recurrent depth ($R = 3$, which our ablation found unnecessary for addition). Trained strictly on 1- to 15-digit sums and multi-tool dialogues, **CBT-18.95M** (`champion_final.pt`) answers 20/20 unseen 16- to 50-digit test sums correctly, routes 26/26 tool cases (25/26 exact arguments, on templates shared with training), and routes 16/18 OOD prompts, of which 8/16 (50.0%) have exact arguments. Recurrent depth gave no measurable benefit for addi-

tion in our 100k ablation. Ultimately, our findings show that while 2D embeddings and character tokenization achieve high performance with exceptional **memory efficiency**, character-by-character recurrence is **not compute-efficient** at scale—meaning current subword LLMs remain well-suited for general cloud AI, whereas CBT is a candidate design for memory-constrained local tool-calling agents and a starting point for studying coordinate-aware architectures, pending the broader tests listed above.

Disclaimer: Use of AI and Compute Infrastructure

Use of AI Assistance: Portions of the experimental engineering, code implementation, data pipeline structuring, and manuscript drafting/formatting in this research were conducted with the assistance of **Gemini 3.8 Flash**. All experimental hypotheses, architectural selections, empirical benchmarks, and final scientific conclusions were verified and validated by the author.

Compute Hardware: All model pretraining, supervised fine-tuning (SFT), and isolated component ablation runs reported in this paper were executed on **Kaggle's NVIDIA Tesla T4 GPUs** (16 GB GDDR6 VRAM, Turing `sm_75` architecture) using PyTorch native FP16 automatic mixed precision (`torch.float16`).

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[2] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[3] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[5] R. Nogueira, Z. Jiang, and J. Lin. Investigating the limitations of transformers with simple arithmetic tasks. *arXiv preprint arXiv:2102.13019*, 2021.

[6] R. Shen, S. Bubeck, R. Eldan, Y. T. Lee, Y. Li, and Y. Zhang. Positional description matters for transformers arithmetic. *arXiv preprint arXiv:2311.14737*, 2023.

[7] S. Jelassi, S. d'Ascoli, C. Domingo-Enrich, Y. Wu, Y. Li, and F. Charton. Length generalization in arithmetic transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[8] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, S. Welleck, P. West, C. Bhagavatula, R. Le Bras, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[9] M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[11] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] A. Giannou, S. Rajput, J.-y. Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*, pages 11398–11442, 2023.

[13] S. McLeish, A. Bansal, A. Stein, N. Jain, J. Kirchenbauer, B. R. Bartoldson, B. Kailkhura, A. Bhatele, J. Geiping, A. Schwarzschild, and T. Goldstein. Transformers can do arithmetic with the right embeddings. In *International Conference on Machine Learning (ICML)*, 2024.

[14] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, and S. Reddy. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[15] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[16] A. Ruoss, G. Delétang, T. Genewein, J. Grau-Moya, R. Csordás, M. Bennani, S. Legg, and J. Veness. Randomized positional encodings boost length generalization of transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[17] Y. Zhou, U. Alon, X. Chen, X. Wang, R. Agarwal, and D. Zhou. Transformers can achieve length generalization but not robustly. *arXiv preprint arXiv:2402.09371*, 2024.

[18] S. Duan and Y. Shi. From interpolation to extrapolation: Complete length generalization for arithmetic transformers. *arXiv preprint arXiv:2310.11984*, 2023.

[19] Y. Fan, Y. Du, K. Ramchandran, and K. Lee. Looped transformers for length generalization. *arXiv preprint arXiv:2409.15647*, 2024.